

An Evidence–Belief–Recommendation–Trust Architecture for Daily Trading Recommendations and a One-Shot Refactor Plan for `hwloser/trade`

Executive summary

This manuscript specifies an Evidence→Belief→Recommendation→Trust (EBRT) architecture for a TradeDB-style daily recommendation system and then maps it to a one-shot refactor plan for the `hwloser/trade` repository. EBRT's goal is not merely to output a “confidence number”, but to make *every recommendation auditable*: which evidence mattered, how beliefs changed, why the decision was taken, and how much the system should be trusted *today* given data freshness, model calibration, pipeline health, and out-of-distribution risk.

I recommend EBRT because it separates concerns that are often conflated: (i) **Evidence** is raw or structured observations with provenance; (ii) **Belief** is a time-evolving state over symbols and horizons (multi-step forecasts), updated by attention-style weighting and residual (ResNet-like) state updates; (iii) **Recommendation** is an optimisation/ranking layer that respects risk constraints and portfolio rules; (iv) **Trust** is a *meta-model* and gate that is grounded in calibration (e.g., Brier score), reliability, observability, and outcome-based reward/penalty feedback.

Crucially, the *core algorithm need not be a neural network*: “attention” can be implemented as similarity-plus-reliability weighting. However, EBRT naturally supports neural components (e.g., Transformer embeddings for articles, multi-head attention for evidence fusion). Backpropagation updates **parameters** of encoders, attention scorers, predictors and calibrators; it does not “backprop into the past” to change recorded beliefs. Instead, beliefs are updated online by residual rules, while parameters are updated offline/rolling via standard supervised learning on realised outcomes.

In `hwloser/trade`, many key ingredients already exist (Bronze/Silver/Gold sentiment pipeline, feed scoring, DataGateway with gap filling, event bus with persistent event_log, job registry, and an ops-oriented web API with DAG runtime and “Today/Picks/Pipeline” endpoints). The refactor consolidates “too many commands” into a single node-based engine, reduces schema sprawl into EBRT core objects, and standardises observability so “trust” becomes a measurable, testable property rather than a vibe. Existing web endpoints can be preserved while their payloads become EBRT traces. ¹

Background and related work

EBRT draws on four strands of primary literature: belief revision, attention and residual updates, denoising and drift detection, and ML observability and calibration.

Belief revision. The AGM framework formalises how belief sets should be contracted or revised under new information, introducing postulates and representation theorems for rational belief change. ²

EBRT is not a logical belief-set system, but the separation between “current beliefs” and “incoming evidence”, plus explicit conflict handling, is AGM-inspired.

State-space filtering. Kalman filtering formalises sequential belief updates as state estimation from noisy observations, with explicit covariance propagation. ³ EBRT uses Kalman-style updates as an optional denoising and smoothing mechanism for continuous signals (e.g., daily sentiment aggregates, volatility regimes).

Attention and residual updates. The Transformer introduces scaled dot-product attention and multi-head attention as a general relevance-weighted aggregation mechanism. ⁴ Residual networks show that learning residual functions around an identity mapping stabilises optimisation of deep systems. ⁵ EBRT uses “attention” as a *design pattern* (relevance weighting) and “residual updates” as a *belief update rule* (new belief = old belief + gated increment). Neither choice forces the system’s core to be a neural network.

Denoising. Denoising autoencoders establish the principle that representations can be made robust by learning to reconstruct clean inputs from corrupted versions. ⁶ EBRT treats denoising as a layered discipline: data-level (deduplication, spam filtering), feature-level (EMA/Kalman smoothing), and representation-level (DAE for embeddings or factor vectors).

Event studies and catalysts. Event studies estimate the market impact of events by measuring abnormal returns around event windows. This is foundational for validating whether extracted “events” in text meaningfully move prices. ⁷ EBRT uses event-study logic to (a) identify catalysts, and (b) reward/penalise evidence sources and influence signals based on ex-post realised impact quality.

Influence diffusion. Influence maximisation formalises how actions propagate through networks under diffusion models like independent cascade and linear threshold. ⁸ EBRT borrows the modelling stance (influence is measurable and propagates) but applies it to information cascades across feeds/influencers and their market impact.

Calibration, “confidence”, and trust. A system’s confidence is useful only if it is calibrated. Modern neural networks can be miscalibrated; temperature scaling is an effective post-hoc calibrator in many settings. ⁹ For probabilistic predictions, the Brier score is a proper scoring rule that rewards honest probabilities and penalises overconfidence. ¹⁰ EBRT’s Trust layer is explicitly scoring-rule-driven, enabling the reward/penalty loop you asked for (high trust but wrong → penalise; low trust but right → reward).

Observability and ML-ops. The ML Test Score proposes a rubric of tests and monitoring practices for production ML and technical debt reduction. ¹¹ Hidden technical debt in ML systems highlights issues like entanglement, hidden feedback loops, and configuration debt—exactly what “command sprawl + ad-hoc DAG + too many tables” tends to create. ¹² Model Cards formalise documentation of model behaviour, evaluation, and limitations—EBRT generalises this into “Recommendation Cards” backed by traces. ¹³ Drift detection can be grounded in distribution tests such as maximum mean discrepancy (MMD). ¹⁴

Two clarifications you explicitly asked:

- **Is confidence “causal”?** No. Confidence is typically about *predictive uncertainty / reliability*. $p(y \mid x)$ is not $p(y \mid do(x))$. EBRT treats confidence/trust as *epistemic* properties, not causal claims. Causal claims require stronger assumptions and designs than EBRT usually has.

- **Is KG “causal”?** Not necessarily. A knowledge graph can encode causal relations, but most mined relations are associative unless validated under causal methodology. EBRT treats KG edges as *evidence-backed associations with confidence* unless explicitly established otherwise. Your repo’s KG tables already store edge confidence/sample_count, which aligns with this view. ¹⁵

Repository scan of `hwloser/trade`

This section summarises what the repo already does (and why it feels “too many commands, too many tables”), focusing on elements relevant to EBRT.

Operational web console and endpoints. The FastAPI backend provides endpoints for DAG browsing and runtime, event/workflow traces, model registry, data health, backups, and also “today-page” and “signals-page” which are close to your desired product surface. ¹ The API already supports rerunning DAG nodes in different modes (self/upstream/full), and streams event/runtime updates via server-sent events. ¹ This is an excellent base for EBRT observability.

Event bus + DAG bootstrap. The repo implements an in-process pub/sub EventBus with SQLite persistence (`event_log`) and can bootstrap handlers from the `pipeline_dag` table instead of hardcoded registration. ¹⁶ This is already the skeleton of the “execution plane” I recommended earlier—your pain is that jobs still call commands, and the conceptual DAG is less usable than it should be.

Job registry and coupling to CLI. Jobs are registered as plain callables (fetch/compute/train) and include sentiment, events, features build, model train, etc. ¹⁷ However, several jobs call CLI entrypoints internally (e.g., a sentiment job calls `trade_py.cli._sentiment.main`). ¹⁷ DataGateway also triggers the sentiment pipeline using the CLI entrypoint to ensure gold exists. ¹⁸ This is a direct source of “command sprawl” and makes the DAG feel like “a wrapper over commands” instead of a first-class engine.

Bronze/Silver/Gold for text and events. The data pipeline includes ingestion, enrichment, aggregation—matching a Bronze/Silver/Gold pattern. Ingestion persists and deduplicates articles and tracks coverage in a pipeline DB. ¹⁹ Enrichment supports incremental processing via content hashes (a de-facto cache). ²⁰ Aggregation computes daily symbol-level metrics, including signal strength and noise penalty. ²¹ This is already “Evidence → aggregated evidence” and fits EBRT’s Evidence layer.

Existing denoising is present but implicit. You said you “didn’t see denoising algorithms”. The repo does include *rule-based noise estimation* at the factor level: `noise_score()` penalises short texts, repeated punctuation, uppercase ratio, and “clickbait words”, and aggregation applies a noise penalty. ²² The LLM-assisted enricher is gated by a confidence threshold and falls back to base methods when “LLM confidence” is low. ²³ That is denoising by gating—just not formalised as “denoising module”.

Feed scoring. The repo computes feed scores across dimensions such as coverage, uniqueness, signal density, reliability and timeliness, and stores structured scoring. ²⁴ The `FeedScore` structure already resembles EBRT’s “SourceTrust”. ²⁵

DataGateway with gap filling and reporting. There is already a “unified read-through data access layer” that tries local first, attempts gap fill, degrades explicitly, and records “data_repair_runs” and “data_gaps” in SQLite. ¹⁸ This is a strong foundation for EBRT’s FreshnessStatus and operational Trust gating.

Core database schema sprawl. `TradeDB` defines many tables: `signals`, `event_log`, `job_runs`, `instruments`, `sync_state`, `planned_events` + `agenda`, `pipeline_dag`, `factors` + `factor_registry`, `model_registry`, multiple evaluation tables, and KG tables. ¹⁵ Migrations show repeated evolution and seeding of DAG rows and schema features. ²⁶ Your “tables are too many” complaint is valid: the schema currently mixes at least five conceptual concerns (ops, research evaluation, KG, forecasting signals, UI caching) into one meta-DB. ¹⁵ EBRT will consolidate them around core objects and make the rest explicitly derived.

Recommendation logic exists but is not EBRT-traced. `TradeDB.signal_recommend` implements a recall → coarse rank → fine rank pipeline that blends `model_score`/`window_score`/`event_kg_score` and penalises risk and sector concentration. ¹⁵ This is already a Recommendation layer, but it lacks a formal “RecommendationTrace” object linking back to evidence and belief transitions.

Model inference and evaluation exist. The online inference service loads active models from `model_registry`, retrieves latest factors, and outputs `model_score` and `model_risk`. ²⁷ The evaluation service computes rank IC, hit rates, sector concentration, Brier score for risk models, calibration bins, and gates operational vs research readiness. ²⁸ This is already part of EBRT’s Trust layer—again, it just isn’t explicitly packaged as “Trust”.

CLI command sprawl is real. The CLI structure contains many domain subcommands (e.g., `event`, `run`, `model`, `factor`, etc.), and some commands call other commands. ²⁹ The web backend even triggers CLI workflows in a background thread. ¹ This is exactly the anti-pattern Sculley et al. warn about: entanglement and configuration/technical-debt growth in ML systems. ¹²

What should *not* be dictated by the current repo

You asked that the paper not be “over-influenced” by `trade`. I agree—EBRT is a generic architecture. The repo scan is treated as a concrete baseline and integration target, not a conceptual constraint. EBRT’s specification below stands independently; the mapping section later shows how to converge the repo toward EBRT without losing working functionality.

EBRT architecture and algorithms

The goal and “how to use it”

EBRT’s goal is: **each day, produce a small set of actionable recommendations with quantified trust and an auditable trace from evidence to decision.**

“Using it” means:

1. Ingest evidence continuously (articles, price data, calendars, etc.).
2. Update beliefs incrementally (per symbol, per horizon).
3. At a decision time (e.g., market open, market close), compute recommendations from beliefs.
4. Publish recommendations along with trust scores and traces.
5. When outcomes arrive (realised returns, drawdown events), update model parameters (offline) and update trust/reliability weights (online), closing the reward/penalty loop.

This is not “just a confidence score”. Trust is the system’s *operational permission to act*, grounded in evidence quality, calibration, and observability.

A seven-layer EBRT design

The repo already hints at this layering in docs and the web UI plan. 30 31 EBRT makes it explicit:

1. **Source layer:** market data, articles, platform posts, economic calendars, reference datasets.
2. **Ingestion layer:** batch + stream ingestion to Bronze; deduplication; provenance.
3. **Evidence layer:** enrichment to Silver (entities, sentiment, events, influencer metadata).
4. **Belief layer:** a persistent BeliefState per symbol and horizon, updated incrementally via attention-style weighting and residual updates (ResNet-like).
5. **Decision layer:** recommendation/reporting logic that optimises ranking under constraints (risk, sector diversification, turnover).
6. **Trust layer:** calibration, gates, freshness, drift, and traceability; decides whether to act, degrade, or block.
7. **Experience layer:** human-readable explanation + UI; outcome feedback attaches reward/penalty to sources and models.

A concise architecture diagram:

```
flowchart TB
    subgraph L1["Source layer"]
        S1["Prices & OHLCV"]
        S2["Fund flow / fundamentals / macro"]
        S3["Articles / socials / influencers"]
        S4["Calendar & planned events"]
    end

    subgraph L2["Ingestion layer"]
        I1["Batch ingest (daily)"]
        I2["Stream ingest (near real-time)"]
        I3["Dedup + normalise + provenance"]
    end

    subgraph L3["Evidence layer"]
        E1["ArticleEvent extraction (Silver)"]
        E2["InfluenceSignal detection"]
        E3["Symbol-day aggregation (Gold)"]
        E4["KG relations + propagation candidates"]
    end

    subgraph L4["Belief layer"]
        B1["BeliefState per (symbol, horizon)"]
        B2["Attention scoring + reliability weights"]
        B3["Residual belief update + decay/smoothing"]
    end

    subgraph L5["Decision layer"]
        D1["Rank/optimize: picks, sizing, risk"]
        D2["Scenario/multi-horizon outputs"]
    end

    subgraph L6["Trust layer"]
```

```

    T1["FreshnessStatus + data gaps"]
    T2["Calibration (Brier), drift (MMD)"]
    T3["Quality gates + lineage + trace"]
end

subgraph L7["Experience layer"]
    X1["Today / Picks / Pipeline UI"]
    X2["RecommendationTrace (human-readable)"]
    X3["Outcome feedback (reward/penalty)"]
end

S1-->I1-->E3-->B1-->D1-->X1
S3-->I2-->E1-->B2-->B1
S3-->E2-->B2
S4-->I1-->E1
E4-->B2
B1-->T3-->X2
T1-->T3
T2-->T3
D1-->T3
X3-->B2
X3-->T2

```

Where “attention” lives, and whether it must be a Transformer

You asked several times “attention is where?” and “multi-head attention is Transformer’s strength—do we need Transformer as core?”.

EBRT’s answer is precise:

- **Attention in EBRT is the Belief update aggregator mechanism.** It is the mathematical rule that tells the system how much each new evidence item should change the belief.
- **Attention does not require a Transformer.** You can compute attention weights with:
 - a simple similarity function between evidence and the current context (information-coefficient-like similarity), plus reliability biases; or
 - a learned neural attention scorer with trainable queries/keys/values (Transformer-style), trained by backprop.

Transformer’s “multi-head attention” is one implementation of the general idea “relevance-weighted aggregation”. ⁴ EBRT uses the general idea and makes it *observable and auditable*.

A practical symbolic-attention definition (no NN required):

- Let e index evidence items relevant to symbol s at time t .
- Let x_e be a feature vector for evidence e (e.g., sentiment strength, event magnitude, novelty, source quality, KG hop score).
- Let $q_{s,t}$ be a “query” vector representing the *current belief context* for s (e.g., regime, volatility, recent news volume).
- Define an attention logit:

$$\ell_e = \frac{q_{s,t}^\top x_e}{\sqrt{d}} + b_{\text{source}(e)} + b_{\text{type}(e)} - \lambda \cdot \text{Noise}(e)$$

- Convert to weights:

$$w_e = \text{softmax}(\{\ell_{e'}\})_e$$

- Compute the evidence message:

$$m_{s,t} = \sum_e w_e \cdot v_e$$

where v_e is the “value” contribution vector (e.g., predicted return shift per horizon).

This is “attention” even without backprop.

If you later want Transformer-style learning, you replace $q_{s,t}$, x_e , and v_e with learned projections $QK^\top$ and value matrices. 4

Residual (ResNet-style) belief updates

You asked “remember residual networks” and “if we have reward/penalty, is resnet update still useful?”.

Residual updates are useful regardless of reward/penalty because they solve a different problem: **stability of state updates under noisy inputs**. Residual learning reformulates updates around “small corrections” rather than “total overwrites”. 5

Let belief state $b_{s,t}$ be a vector containing multi-horizon predictions and uncertainty summaries:

$$b_{s,t} = \left(\mu_{1d}, \mu_{5d}, \mu_{20d}, \sigma_{1d}, \sigma_{5d}, \sigma_{20d}, p_{5d}^\downarrow, p_{5d}^\uparrow, \dots \right)$$

Define the belief update as:

$$b_{s,t+1} = b_{s,t} + \alpha_{s,t} g_{s,t} \odot m_{s,t}$$

- $\alpha_{s,t} \in [0, 1]$ is a step size (how quickly beliefs change today). - $g_{s,t} \in [0, 1]^k$ is a *gate* per belief dimension; it suppresses updates when evidence is weak or conflicting. - $m_{s,t}$ is the attention-aggregated evidence message.

This is the ResNet pattern: **identity + residual**. 5

Reward/penalty affects how you learn or tune α , g , and the attention logits. Residual structure remains valuable because it prevents overreaction to single noisy evidence items.

Denoising strategies and whether “GPT-attention removes noise”

You asked: “If I use GPT attention, do I still need denoising? Do I need denoising per data source?”

Answer: **Yes, you still need denoising**, for three reasons:

1. Attention can *attend to noise* if noise is correlated with targets (e.g., sensational headlines correlate with volatility but not direction). Learned attention is not a guarantee of robustness.

2. In finance/text, adversarial and spam patterns are common; they exploit model heuristics.
3. Denoising is not just filtering; it is **variance reduction** and **stability management**, which attention alone does not solve.

EBRT uses a layered denoising stack:

Data-level denoising (Bronze): - Deduplicate by content hash and canonical URL; your ingest already does dedup and tracks coverage. ¹⁹ - Source health monitoring: empty days, duplicate rate, ingest errors; your evaluation service already computes these. ²⁸

Evidence-level denoising (Silver): - Noise score & novelty score: your `base_factors` already computes `noise_score()` and `novelty_score()` (rule-based). ²² - LLM gating: your enricher selects LLM outputs only above a confidence threshold and otherwise falls back to base logic. ²³

Aggregate-level denoising (Gold): - Temporal smoothing: EMA/Kalman on daily symbol sentiment/event scores. Kalman filtering gives a principled update with uncertainty. ³ - Reliability-weighted aggregation: weights incorporate feed scoring and source health. Your feed scorer already estimates reliability-like components. ²⁴

Representation-level denoising: - Denoising autoencoder (DAE) on embeddings or factor vectors: learn to reconstruct “clean” representations from corrupted inputs. ⁶

Should denoising vary by data source? **Yes**—not in philosophy, but in parameterisation. Different sources have different noise distributions; your repo already distinguishes sources in bronze and evaluates them separately. ²⁸ EBRT makes this explicit by learning/maintaining per-source reliability priors and per-source denoising policy.

Influence and catalyst detection

You proposed “influence” (a big V on a platform triggers volatility) and worried about noise.

EBRT models this with an `InfluenceSignal` object that is created when:

1. a message/post is attributed to an actor (influencer id);
2. the message is linked to symbols/entities (NER + entity linking);
3. the market reaction is abnormal under an event-study window (abnormal return/volume). ⁷
4. the reaction propagates across other channels (reposts, other news outlets), analogous to diffusion. ⁸

Formally, for a post p at time τ affecting symbol s :

- Abnormal return:

$$AR_{s,\tau+\Delta} = R_{s,\tau+\Delta} - \hat{R}_{s,\tau+\Delta}$$

where $\hat{R}$ is a baseline model (market model or sector model). Event studies provide standard tooling for this. ⁷

- Catalyst score:

$$\text{Catalyst}(p, s) = \underbrace{z(|CAR|)}_{\text{impact}} \cdot \underbrace{\text{Novelty}(p)}_{\text{not a repeat}} \cdot \underbrace{\text{Propagation}(p)}_{\text{diffusion}} \cdot \underbrace{\text{ActorTrust}(a)}_{\text{actor reliability}}$$

where CAR is cumulative abnormal return over a window. ActorTrust is *learned* over time via reward/penalty (below).

This gives you the mechanism you wanted: a “big V” can matter—but only if the system measures consistent impact and downgrades noisy actors.

Backpropagation and “how iteration happens”

You asked: “I didn’t see backprop in the paper—how can it iteratively update?”

EBRT has *two kinds of iteration*:

Online state updates (no backprop): - Evidence arrives → belief updates via residual rules. - This is the operational loop.

Offline/rolling parameter learning (with backprop or other learners): - Once outcomes are known (realised returns, drawdowns), update: - the prediction model parameters (NN backprop or tree boosting); - the attention scorer parameters (if learned); - the trust calibrator parameters (temperature scaling / logistic calibration); - source/influencer reliability weights (online learning).

Transformers require backprop during training; EBRT allows but does not force a Transformer-core. ⁴
Your repo currently appears to use scikit-style models (joblib-loaded predictors) and computes calibration metrics. ²⁷ ²⁸ That’s valid: the system can iterate by retraining weekly/daily and by updating trust weights daily.

Can trust be updated by backprop? **Partially**: - If trust is derived from probabilistic outputs, you can learn a calibration model with gradients (e.g., temperature scaling, logistic calibration). ³² - But many trust components (data freshness, pipeline errors, provenance existence) are **structural** and should remain non-learned, because they are factual constraints.

Articles, embeddings, and whether factors should be embedded

You asked: - “Can I embed articles and let the model learn, with sentiment as a factor?” - “Do factors need embedding?”

EBRT’s recommended stance:

Article embeddings are appropriate. Use a Transformer encoder to produce an embedding for each article/post, then learn: - (a) classification heads for event_type, sentiment polarity, entity links, credibility; and/or - (b) directly learn predictive contributions v_e for horizons. ⁴

However, you should **not** rely on the embedding alone to remove noise. Add explicit denoising and reliability gating; denoising autoencoders are one principled tool. ⁶

Factors generally do **not** need embedding if they are numeric and well-defined (momentum, volatility, fund flow). They can be fed directly into tree models or linear models. Your repo uses a factor store in SQLite and serves factors to inference. ¹⁵ ²⁷

Factor embeddings can help in two cases: 1. **Categorical factors** (event types, sector IDs, news source IDs) benefit from embeddings. 2. **Meta-learning** where you treat “factor identity” as a token and learn interactions; this is heavier and rarely needed early.

EBRT’s principle: keep factors interpretable and numeric by default; embed only categorical or high-cardinality IDs (source, actor, event_type).

Data model, observability, evaluation, and trust calibration

Core EBRT objects and SQL-like schemas

Below are the *core EBRT tables* (designed to replace “too many tables” with a coherent set). I keep them SQL-like; storage can be SQLite for metadata/state and Parquet/DuckDB for large evidence payloads, consistent with the repo’s existing direction (SQLite + parquet + DuckDB reads). 15 28

ArticleEvent (Silver, per document)

```
CREATE TABLE article_event (
  evidence_id      TEXT PRIMARY KEY,
  source_name      TEXT NOT NULL,
  source_family    TEXT,
  provider_kind    TEXT,
  published_at     TEXT NOT NULL,
  fetched_at       TEXT,
  url              TEXT,
  title            TEXT,
  content_hash     TEXT UNIQUE,
  language         TEXT,
  entities_json    TEXT,      -- linked symbols/sectors/people
  event_type       TEXT,      -- e.g., earnings_beat, policy_positive
  sentiment        REAL,      -- [-1,1]
  magnitude        REAL,      -- [0,1] or signed
  novelty_score    REAL,
  noise_score      REAL,
  llm_confidence   REAL,
  embedding_ref    TEXT,      -- pointer to vector store / parquet
  raw_json         TEXT
);
```

InfluenceSignal (actor/platform impact, derived)

```
CREATE TABLE influence_signal (
  influence_id     TEXT PRIMARY KEY,
  evidence_id      TEXT REFERENCES article_event(evidence_id),
  actor_id         TEXT NOT NULL,
  platform         TEXT NOT NULL,
  actor_features_json TEXT,    -- follower count, verification, etc.
  symbol           TEXT,
```

```

event_window      TEXT,      -- e.g. "0..2d"
abnormal_return    REAL,
abnormal_volume_z  REAL,
propagation_score  REAL,
catalyst_score     REAL,
estimated_noise    REAL,
created_at         TEXT NOT NULL
);

```

Evidence (unifying wrapper for all evidence modalities)

```

CREATE TABLE evidence (
  evidence_id      TEXT PRIMARY KEY,
  kind             TEXT NOT NULL,      -- article, price, kg, macro,
influence
  symbol          TEXT,               -- nullable for market-level
  ts              TEXT NOT NULL,
  value_json       TEXT NOT NULL,      -- structured payload
  provenance_json  TEXT NOT NULL,      -- source, pipeline, hashes
  reliability      REAL NOT NULL DEFAULT 0.5,
  noise            REAL NOT NULL DEFAULT 0.0
);

```

BeliefState (per symbol, per horizon)

```

CREATE TABLE belief_state (
  symbol          TEXT NOT NULL,
  as_of_date      TEXT NOT NULL,
  horizon         INTEGER NOT NULL,    -- days: 1,5,20...
  mu_return       REAL,               -- expected return (% or log)
  sigma_return    REAL,               -- uncertainty
  p_up            REAL,               -- optional
  p_down          REAL,               -- optional
  regime          TEXT,               -- optional categorical
  state_json       TEXT,               -- extra latent state
  updated_at      TEXT NOT NULL,
  PRIMARY KEY (symbol, as_of_date, horizon)
);

```

AttentionScore (observable attention weights)

```

CREATE TABLE attention_score (
  trace_id        TEXT NOT NULL,
  evidence_id      TEXT NOT NULL,
  symbol          TEXT NOT NULL,
  as_of_date      TEXT NOT NULL,

```

```

horizon            INTEGER NOT NULL,
logit              REAL NOT NULL,
weight            REAL NOT NULL,
reliability_used   REAL NOT NULL,
noise_used         REAL NOT NULL,
head              TEXT DEFAULT 'default',
PRIMARY KEY (trace_id, evidence_id, horizon)
);

```

BeliefTransition (residual update record)

```

CREATE TABLE belief_transition (
  trace_id          TEXT PRIMARY KEY,
  symbol            TEXT NOT NULL,
  as_of_date        TEXT NOT NULL,
  horizon           INTEGER NOT NULL,
  prev_belief_json  TEXT NOT NULL,
  message_json      TEXT NOT NULL,      -- m_{s,t}
  gate_alpha        REAL NOT NULL,
  gate_vector_json  TEXT,
  next_belief_json  TEXT NOT NULL,
  conflict_flag     INTEGER NOT NULL DEFAULT 0,
  decay_tau_days    REAL,
  created_at        TEXT NOT NULL
);

```

Recommendation (decision output)

```

CREATE TABLE recommendation (
  rec_id            TEXT PRIMARY KEY,
  as_of_date        TEXT NOT NULL,
  symbol            TEXT NOT NULL,
  action            TEXT NOT NULL,      -- buy/sell/watch/avoid
  score             REAL NOT NULL,
  expected_return_5d REAL,
  risk_5pct         REAL,
  position_weight   REAL,
  horizon_set_json  TEXT,               -- multi-horizon summary
  reason_codes_json TEXT,
  created_at        TEXT NOT NULL
);

```

QualityReport (daily gate + metrics)

```

CREATE TABLE quality_report (
  as_of_date        TEXT PRIMARY KEY,

```

```

operational_status TEXT NOT NULL,      -- ok/degraded/blocked
research_status    TEXT NOT NULL,      -- ok/partial/blocked
metrics_json       TEXT NOT NULL,
reasons_json       TEXT NOT NULL,
created_at         TEXT NOT NULL
);

```

RunRecord (unify job_runs + event_log conceptually)

```

CREATE TABLE run_record (
  run_id          TEXT PRIMARY KEY,
  kind            TEXT NOT NULL,        -- node_run, workflow, retrain
  node_id         TEXT,
  status          TEXT NOT NULL,
  started_at      TEXT NOT NULL,
  completed_at    TEXT,
  elapsed_ms      INTEGER,
  input_signature TEXT,
  output_artifacts_json TEXT,
  error           TEXT
);

```

FreshnessStatus

```

CREATE TABLE freshness_status (
  dataset         TEXT NOT NULL,
  item_key        TEXT NOT NULL,
  as_of_date      TEXT,
  lag_days        INTEGER,
  coverage_pct    REAL,
  degraded        INTEGER NOT NULL DEFAULT 0,
  reason_code     TEXT,
  updated_at      TEXT NOT NULL,
  PRIMARY KEY (dataset, item_key)
);

```

RecommendationTrace (the EBRT spine)

```

CREATE TABLE recommendation_trace (
  trace_id        TEXT PRIMARY KEY,
  rec_id          TEXT NOT NULL REFERENCES recommendation(rec_id),
  belief_trace_id TEXT NOT NULL REFERENCES belief_transition(trace_id),
  trust_json      TEXT NOT NULL,        -- components + scalar
  narrative_text   TEXT,                -- human-readable
);

```

```
created_at          TEXT NOT NULL
);
```

Trust is not “only confidence”: a measurable definition

Trust in EBRT is a **structured vector**, then optionally collapsed to a scalar.

I recommend modelling trust as:

$$\text{Trust} = (T_{\text{fresh}}, T_{\text{evidence}}, T_{\text{model}}, T_{\text{calib}}, T_{\text{drift}}, T_{\text{ops}}, T_{\text{explain}})$$

- T_{fresh} : data freshness/coverage (leverages DataGateway gap reports and data health scans). Your repo already tracks data gaps and repairs and exposes data health. ¹⁸ ¹
- T_{model} : model validity (rank IC, hit rate, etc.). Your evaluation computes these and stores them. ²⁸
- T_{calib} : calibration quality (Brier score, reliability curve). Your evaluation already computes Brier and calibration bins for the risk model. ²⁸
- T_{ops} : pipeline health (job errors, event bus pending, gate status). Your repo tracks job_runs and event_log and synthesises workflow details. ¹⁵ ¹
- T_{explain} : trace completeness (do we have evidence links? belief transition record? top contributors?).

A scalar trust score can be:

$$T^{\setminus *} = \sigma(w^{\top} \phi(\text{Trust}))$$

but the UI should still show components (“why trust is low today”).

How do we “trust it”?

EBRT says: you trust the system when (a) it is calibrated, (b) it is observable, (c) it can explain itself, and (d) it has built in gates to refuse action when conditions are poor. This aligns with ML Test Score’s emphasis on tests and monitoring for production readiness. ¹¹

Reward/penalty feedback when trust is wrong

You asked: “If trust score is high but reality is bad—punish. If trust is low but reality is good—reward.”

This is exactly what **proper scoring rules** do. The Brier score, introduced by Brier (1950), penalises squared error between predicted probability and outcome. ¹⁰

For a binary event $y \in \{0, 1\}$ (e.g., “return_5d > 0” or “drawdown beyond -5%”), with predicted probability p :

$$\text{Brier}(p, y) = (p - y)^2$$

- If trust/confidence is high ($p \approx 1$) but outcome is 0 → penalty near 1.
- If trust is low ($p \approx 0$) but outcome is 1 → penalty near 1.
- Accurate uncertainty estimates are rewarded.

EBRT uses this in two places:

1. **Calibration model update:** learn a mapping from raw model outputs to calibrated probabilities (e.g., temperature scaling). Guo et al. show this is important because modern models can be miscalibrated. ⁹
2. **Source/actor reliability update:** maintain per-source reliability weights and update them online via multiplicative/exponential weighting using loss (Brier or log loss). This is your “punish/reward” in a mathematically grounded way.

Drift detection with MMD

Finance is non-stationary. EBRT’s Trust layer includes a drift component:

- Choose a feature set X (e.g., factor distributions, embedding distributions).
- Compare recent window X_{recent} to a reference window X_{ref} .
- Use MMD as a distribution discrepancy statistic. Gretton et al. provide the classic kernel two-sample test formulation. ¹⁴

When drift is high, reduce T_{drift} and/or widen uncertainty (σ) so the system becomes less confident.

Observability and three web panels

Your repo already aligns with a “Today / Picks / Pipeline” orientation via endpoints and UI planning. ¹

³¹ EBRT formalises what each panel must show:

Panel: Today

Purpose: operational readiness + market snapshot + “should I trust outputs today?”

```
+-----+
+
| TODAY (as of YYYY-MM-DD)                                TRUST: 0.78 [OK /
DEGRADED] |
+-----+-----+
+-----+
| Data Freshness      | Model Health      |
Alerts                |                  |
| Kline: OK lag=0d    | RankIC_5d: 0.031    | 2 DAG nodes
failed                |                  |
| Sentiment Gold: OK  | Risk Brier: 0.084   | 1 source
degraded              |                  |
| Events: PARTIAL     | Calibration: stable | Drift:
medium                |                  |
+-----+-----+
+-----+
| Top 5 Picks (headline only)                                |
| 1) 600xxx.SH score=0.91 trust=0.82                          |
[NEW]                  |
|
| 2) ...
```

```
|
+-----+
+
```

Panel: Picks

Purpose: actionable list + evidence trace + belief transition.

```
+-----+
+
| PICKS (Top 50)    Filters: [trust>0.6] [sector cap=3]
| [horizon=5d]      |
+-----+-----+
+-----+
| # | Symbol | Act  | Score | Trust | Expected 5d / Risk | Why (top
3) |
+-----+-----+
+-----+
| 1 | 600xxx | BUY  | 0.91  | 0.82  | +2.3% / 0.06      | event+,
sent+, |
|   |        |      |       |       |                   | kg-
path   |
|   | Trace -> | Evidence weights (attention) + Belief delta
(residual) |
+-----+
+
```

Panel: Pipeline

Purpose: make the execution plane debuggable (what ran, what failed, what is stale).

```
+-----+
+
| PIPELINE / DAG Runtime (last 200
events) |
+-----+-----+
+-----+
| Node                | Stage | Status | Last run | Error / Root
cause |
+-----+-----+
+-----+
| sentiment_fetch     | fetch | OK      | 22:01    |
- |
| sentiment_gold      | fetch | ERROR   | 22:10    | parquet write
failed |
| event_extract       | compute | PENDING | -        | waiting
upstream |
+-----+
+
| Workflow Trace: root_event_id=12345 status=partial (rerun
upstream?) |
```


This approach matches the repo's existing capabilities: DAG runtime, workflows, reruns, and data health snapshots. ¹

You want a “big rewrite”: fewer commands, a usable DAG, fewer tables. I recommend a one-shot refactor that preserves working behaviour while re-platforming around EBRT.

Replace “commands calling commands” with a single node engine whose nodes are importable functions with explicit inputs/outputs and explicit state.

This directly addresses the repo coupling issues: - Jobs call CLI entrypoints. ¹⁷ - DataGateway triggers CLI. ¹⁸ - Web triggers CLI workflow threads. ¹

I recommend rewriting into four top-level packages (still under `trade_py/` to minimise disruption):

- `trade_py/evidence/` — ingestion + enrichment + aggregation; produces Evidence and ArticleEvent.
- `trade_py/belief/` — belief state store and attention/residual updates; produces BeliefState and BeliefTransition.
- `trade_py/decision/` — recommendation ranking/optimisation; produces Recommendation.
- `trade_py/ops/` — run records, gates, health, UI snapshots, event bus integration.

A mapping table (selected high-impact files):

Current repo file/module	Problem today	EBRT target package
trade_py/data/ pipeline/ingest.py 19	Ingestion is OK but not “Evidence API”	trade_py/evidence/ingest.py
trade_py/data/ pipeline/enrich.py 20	Enrichment cache exists but not formal evidence objects	trade_py/evidence/enrich.py
trade_py/data/ pipeline/aggregate.py 21	Aggregation outputs are not linked to belief transitions	trade_py/evidence/ aggregate.py
trade_py/intelligence/ enricher.py 23	LLM gating exists but is not a reliability component	trade_py/evidence/ semantic.py + trade_py/ops/ reliability.py
trade_py/intelligence/ feed_scorer.py 24	Feed score not integrated into trust	trade_py/ops/ source_trust.py

Current repo file/module	Problem today	EBRT target package
<code>trade_py/bus/</code> <code>__init__.py</code> ¹⁶	Good base; keep, but handlers should run nodes, not commands	<code>trade_py/ops/event_bus.py</code>
<code>trade_py/jobs/</code> <code>__init__.py</code> ¹⁷	Job registry calls CLI internals	Replace with <code>NodeRegistry</code> (nodes call pure functions)
<code>trade_py/db/</code> <code>trade_db.py</code> ¹⁵	Meta DB mixes many concerns	Split into <code>ops_db</code> + <code>ebrt_db</code> tables; keep compat views
<code>trade_web/backend/</code> <code>app.py</code> ¹	UI endpoints already great	Keep endpoints; swap payloads to EBRT trace objects
<code>trade_web/backend/</code> <code>inference.py</code> ²⁷	Inference is factor-based, not belief-based	Move to <code>decision/predictors.py</code> with point-in-time support

Collapsing the “too many tables” problem

Current schema already contains: - ops tables: `event_log`, `job_runs`, `ui_snapshots`, `pipeline_dag`, backups; ¹⁵ - research tables: model eval runs, dataset snapshots, daily gate; ¹⁵ - signal tables: `signals`, `factors`; ¹⁵ - KG tables: nodes/relations/candidates, events/propagations. ¹⁵

EBRT consolidation plan:

1. Keep **ops** tables (`event_log/job_runs/pipeline_dag`) largely intact because the web console relies on them. ¹
2. Introduce EBRT tables (`evidence`, `belief_state`, `belief_transition`, `recommendation`, `recommendation_trace`) and gradually replace `signals` as “derived cache”.
3. Convert existing `signals` into a *view* over `recommendation` and `belief_state` for backward compatibility.

Migration mapping example:

Current table	EBRT replacement	Notes
<code>signals</code> ¹⁵	<code>belief_state</code> + <code>recommendation</code>	Keep <code>signals</code> as derived cache initially
<code>daily_quality_gate</code> ¹⁵	<code>quality_report</code>	Same semantics; rename + extend
<code>source_eval_daily</code> / <code>source_health_daily</code> ¹⁵	<code>source_trust_daily</code> (in ops)	Keep fields; connect to Trust scoring
<code>ui_snapshots</code> ¹⁵	keep	Use for Today/Picks/ Pipeline snapshot caching

A C++ engine role, without forcing it today

Your prompt asks for “C++ engine roles”. The repo is Python, and that’s fine. A C++ engine is optional and should be introduced only if profiling shows a need.

If you do add `trade_cc/engine`, I recommend these roles:

- **trade_cc::state_store**: memory-mapped belief state cache + fast updates.
- **trade_cc::attention**: compute attention logits/softmax and top-k contributions quickly.
- **trade_cc::risk**: portfolio constraints, optimisation and sizing.
- **trade_cc::trace**: compact trace generation for UI (top contributors, belief deltas).
- **trade_cc::ffi**: Python bindings (pybind11), so Python remains orchestration layer.

Python remains the control plane; C++ accelerates hot loops.

Evaluation plan

EBRT evaluation is multi-layered and should match ML Test Score style (tests + monitoring). ¹¹

Unit tests (correctness): - Deterministic evidence parsing and deduplication. - Belief update algebra properties (e.g., weights sum to 1; residual update bounded).

Replay tests (completeness/reproducibility): - Rebuild a week from archived evidence and verify bit-identical belief transitions given fixed model artefacts.

Backtests (effectiveness): - Event-study validation of catalysts. ⁷ - Multi-horizon forecasting metrics (IC, hit rate, MAE).

Calibration metrics (trustworthiness): - Brier score and reliability curves for probabilistic outputs. ³³ - Temperature scaling for calibration. ⁹

Drift detection: - MMD between training and live feature distributions. ¹⁴

A/B and canary: - Run EBRT in shadow mode alongside existing `signals` logic, compare outcomes and trust gating decisions before switching.

Rollout timeline and Gantt

A rollout should preserve operational behaviour while introducing EBRT objects.

```
timeline
  title EBRT refactor rollout
  Phase 1 : Introduce EBRT tables (evidence/belief/recommendation/trace)
  alongside existing signals
    : Add trace writing for current recommendations
  Phase 2 : Replace command-calling-command with Node Engine (jobs -> nodes -
  > bus)
    : Keep web endpoints stable, update payloads gradually
  Phase 3 : Trust calibration + reward/penalty loop (source/actor
  reliability)
```

```

: Add drift detection (MMD) + model cards / recommendation cards
Phase 4 : Deprecate legacy tables and commands
: Optional C++ acceleration if needed

```

```

gantt
  title One-shot refactor plan (no specific constraint)
  dateFormat YYYY-MM-DD
  axisFormat %Y-%m

  section Foundation
  EBRT schemas + compat views           :a1, 2026-03-20, 21d
  NodeRegistry + NodeRunner             :a2, after a1, 21d

  section Engine migration
  Replace CLI-internal calls (jobs->nodes) :b1, after a2, 28d
  EventBus handlers run nodes, not CLI     :b2, after b1, 14d

  section Trust + feedback
  Calibration + scoring-rule feedback      :c1, after b2, 21d
  Source/Influencer reliability learning    :c2, after c1, 21d
  Drift detection (MMD) and alerts          :c3, after c2, 14d

  section UI + observability
  Today/Picks/Pipeline panels from traces :d1, after b2, 21d
  Recommendation cards export              :d2, after d1, 14d

  section Cleanup
  Deprecate legacy CLI commands            :e1, after d2, 14d
  Optional: C++ hot loop acceleration      :e2, after e1, 28d

```

Why this refactor directly addresses your pain points

- **“Commands are too many and too messy.”**

The node engine becomes the single mechanism: CLI becomes thin wrappers over `run_node(node_id, params)`. The web backend no longer shells into CLI. This removes entanglement and aligns with Hidden Technical Debt warnings. ¹²

- **“DAG is not usable and just calls commands.”**

EventBus already boots handlers from `pipeline_dag`. ¹⁶ The refactor makes DAG nodes pure-function execution units with explicit artefacts and traces.

- **“Tables are too many.”**

EBRT reduces the conceptual model to a small set of core objects; ops tables stay, research tables become subordinated to Trust and evaluation.

- **“How do we get trust?”**

Trust is earned by calibration (Brier/reliability), observability (Today/Picks/Pipeline), provenance, and outcome feedback loops. ³⁴ Your repo already has much of the instrumentation (quality gate, data gap logs, model eval) that EBRT formalises. ²⁸ ¹⁸

Final note on “Transformer as core” and multi-choice prediction

You asked if the system “already needs Transformer as core” and whether it can “predict multiple points / multiple choices”.

EBRT supports multi-horizon, multi-scenario predictions by design: - **Multiple horizons:** output heads for $1d$, $5d$, $20d$ (and risk). Your repo already distinguishes return and risk models. ²⁷ - **Multiple choices:** provide scenarios (bull/base/bear) by predicting quantiles or mixture components; these become part of BeliefState and are consumed by Decision layer.

Transformer is beneficial for text understanding and for learned attention, but EBRT does not require a Transformer core. If your first priority is correctness/observability/trust, a hybrid is often superior: keep a robust symbolic EBRT loop, add Transformers for embeddings and extraction, and introduce learned attention once you have stable traces and evaluation. This mirrors the ML Test Score philosophy: build testable and monitorable systems, then add complexity safely. ¹¹

¹ app.py

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_web/backend/app.py

² On the logic of theory change: Partial meet contraction and revision functions | The Journal of Symbolic Logic | Cambridge Core

https://www.cambridge.org/core/journals/journal-of-symbolic-logic/article/abs/on-the-logic-of-theory-change-partial-meet-contraction-and-revision-functions/7ED837BAD5FB6D9A7C77906D73527F9C?utm_source=chatgpt.com

³ A New Approach to Linear Filtering and Prediction Problems | CiNii Research

https://cir.nii.ac.jp/crid/1360855570047666048?utm_source=chatgpt.com

⁴ Provided proper attribution is provided, Google hereby grants permission to

https://arxiv.org/pdf/1706.03762?utm_source=chatgpt.com

⁵ Deep Residual Learning for Image Recognition - Microsoft Research

https://www.microsoft.com/en-us/research/publication/deep-residual-learning-for-image-recognition/?utm_source=chatgpt.com

⁶ Extracting and Composing Robust Features with Denoising

https://www.cs.toronto.edu/~larocheh/publications/icml-2008-denoising-autoencoders.pdf?utm_source=chatgpt.com

⁷ EconPapers: Event Studies in Economics and Finance

https://econpapers.repec.org/RePEc%3Aaea%3Ajeclit%3Av%3A35%3Ay%3A1997%3Ai%3A1%3Ap%3A13-39?utm_source=chatgpt.com

⁸ The paper "Maximizing the Spread of Influence through a Social Network" from 2003, by David Kempe (CUCS PhD '03), Jon Kleinberg, and Eva Tardos has won one of three inaugural SIGKDD Test of Time Awards | Department of Computer Science

https://www.cs.cornell.edu/information/news/newsitem986/paper-maximizing-spread-influence-through-social-network-2003-david?utm_source=chatgpt.com

⁹ ³² On Calibration of Modern Neural Networks

https://proceedings.mlr.press/v70/guo17a.html?utm_source=chatgpt.com

¹⁰ ³³ ³⁴ Verification of Forecasts Expressed in Terms of Probability - ADS

https://ui.adsabs.harvard.edu/abs/1950MWRv...78....1B/abstract?utm_source=chatgpt.com

¹¹ The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction

https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/?utm_source=chatgpt.com

12 **Hidden Technical Debt in Machine Learning Systems**

https://papers.neurips.cc/paper_files/paper/2015/hash/86df7dcfd896fcdf2674f757a2463eba-Abstract.html?utm_source=chatgpt.com

13 **Model Cards for Model Reporting**

https://research.google/pubs/model-cards-for-model-reporting/?utm_source=chatgpt.com

14 **A Kernel Two-Sample Test**

https://www.jmlr.org/papers/v13/gretton12a.html?utm_source=chatgpt.com

15 **trade_db.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/db/trade_db.py

16 **__init__.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/bus/__init__.py

17 **__init__.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/jobs/__init__.py

18 **gateway.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/data/access/gateway.py

19 **ingest.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/data/pipeline/ingest.py

20 **enrich.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/data/pipeline/enrich.py

21 **aggregate.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/data/pipeline/aggregate.py

22 **base_factors.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/intelligence/base_factors.py

23 **enricher.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/intelligence/enricher.py

24 **feed_scorer.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/intelligence/feed_scorer.py

25 **score.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/meta/feed/score.py

26 **migrations.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/db/migrations.py

27 **inference.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_web/backend/inference.py

28 **service.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/evaluation/service.py

29 **main.py**

https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/trade_py/cli/main.py

30 **claude-latest-plan.md**

<https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/docs/claude-latest-plan.md>

31 **web-ui-v3-plan.md**

<https://github.com/Hwloser/trade/blob/8f26a9102c475fa0e502006386e6033ceba18e05/docs/web-ui-v3-plan.md>